

When Agent Context Goes Stale: Incoherence in Volatile Agent Context

Yingying Liu*
The University of Hong Kong
Hong Kong SAR, China

Junzhou Fang*
The University of Hong Kong
Hong Kong SAR, China

Chenxiong Qian†
The University of Hong Kong
Hong Kong SAR, China

Abstract

Modern agents increasingly ground their reasoning in observations returned by tools, such as file contents read from a workspace. However, the data sources underlying these observations may later be modified by users, other agents, or external tools, while the model retains only the stale content in its context window. Existing agent runtimes provide little support for notifying the model that a previously observed fact has become stale, causing agents to reuse outdated observations and make incorrect claims about the current workspace state.

We propose CONCORD, a context coherence framework that maintains the consistency between tool observations in agent context and the mutable sources from which they were derived. CONCORD links each observation to its source, detects source changes, and uses configurable handling policies to update, annotate, or suppress stale context before reuse. CONCORD is applicable across different agent runtimes and external resources, and can be easily extended to new runtime-resource settings. We implement CONCORD as a general framework, and instantiate a concrete use case to assess its effectiveness. We construct ConcordBench, where previously observed file contents become stale after subsequent edits. Across three evaluated frontier models, CONCORD produces answers consistent with the restored workspace state in all evaluated cases under these constructed conditions, matching the oracle on recover count for this benchmark, while using 46.4% fewer tokens than the strongest non-oracle baseline.

1 Introduction

AI agents are increasingly used as tool-using systems that interact with external resources during task execution. Modern agent runtimes, such as OpenClaw [14, 15], OpenCode [16, 17], and OpenHands [26] support built-in tools and extensions based on the Model Context Protocol (MCP) [2] for reading files, executing commands, querying services, and accessing remote resources. As a result, an agent’s context is no longer limited to the initial user prompt or multi-turn dialog history, but also includes tool observations produced during execution. These tool observations are typically stored as

structured execution entries and later projected into LLM messages before each model call.

Tool observations, however, are not live views of external resources. They are snapshots produced at a particular point in the agent’s execution, while the resources they describe may continue to change outside the agent’s view. For example, in multi-agent systems [9], one agent may update a file that another agent has already read; a developer may manually revise a configuration during an agent run; or a database queried through an MCP tool may be modified by an external transaction. If the agent runtime preserves earlier tool results as reusable context without tracking whether their source data has changed, stale observations may continue to be presented to the model as current facts, leading to incorrect actions or hallucinated claims about the environment.

Existing work addresses related consistency problems at either the workspace or retrieval level. Workspace-level systems reduce conflicts through write validation, isolated workspaces, synchronization, or snapshot reconstruction [5, 12, 20, 21, 29]. Memory and retrieval systems use freshness-aware updates, conflict detection, provenance, or temporal supersession to decide which stored or retrieved evidence should be used for generation [8, 13, 24]. These approaches show the importance of freshness and consistency, but they do not repair tool observations after those observations have already been materialized inside an agent’s private prompt context. Consequently, obsolete observations can remain in later prompts and continue to shape model reasoning after their sources have changed.

Our approach. To address this gap, we propose CONCORD, a context coherence framework that maintains the consistency between tool observations in agent context and the mutable sources from which they were derived. At its core, CONCORD uses a runtime- and source-agnostic Context Manager, which tracks tool observations independently of any particular agent framework or external resource. Runtime Adapters connect the manager to different agent runtimes, while Source Adapters capture changes from different mutable resources. CONCORD is therefore easy to extend to new runtimes, tools, and resource types.

We make the following contributions:

- We identify and formally define incoherence in volatile agent context as a coherence problem in tool-using agents, where context entries may lose consistency with the mutable external sources.

*Yingying Liu and Junzhou Fang contributed equally to this work.

†Corresponding author

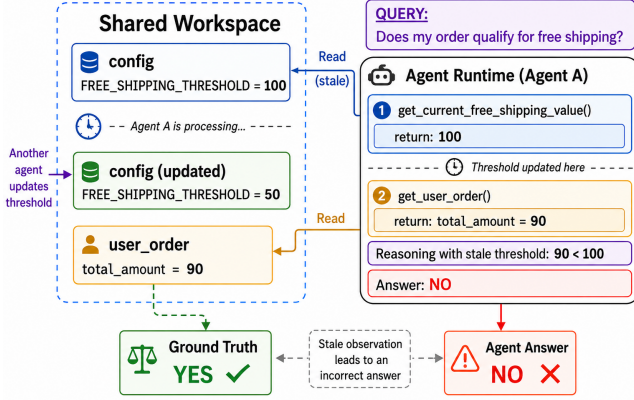


Figure 1. A stale-observation failure in an agent-maintained workspace. After the agent records an old free-shipping threshold in its context, the configuration file is edited outside its view. The agent later answers from the stale observation, producing an answer that contradicts the current workspace state.

- We introduce CONCORD, a runtime- and resource-agnostic framework that tracks source-linked context blocks, detects source changes, and reconciles stale observations through configurable policies.
- We instantiate CONCORD for local file resources and evaluate it on a HotpotQA-derived race benchmark across three frontier language models. Under the constructed race conditions in the 40-task ConcordBench-Pioneer, CONCORD achieves a recover count of 40/40 for each evaluated model, matching the oracle recover-count upper bound on this benchmark while using fewer tokens than the evaluated non-oracle baselines.

2 Motivation

2.1 Example: Stale Context after Workspace Changes

As shown in Figure 1, we illustrate the problem with a controlled agentic coding workflow built using the OpenHands SDK [26]. The agent is asked to maintain a shop project, where free shipping applies when an order total is greater than or equal to a configurable threshold. Initially, the agent reads the project files and observes that `FREE_SHIPPING_THRESHOLD` is set to 100. The runtime then appends the file-reading tool return to the agent’s conversation context. After that, the underlying configuration file is modified outside the agent’s view, lowering the threshold from 100 to 50. When the agent finally answers whether an order totaling 60 qualifies for free shipping, it answers “no” based on the stale threshold value of 100. However, the current workspace state implies the opposite answer: since the threshold is now 50, the order should qualify for free shipping. This example highlights a key limitation of current

agent runtimes: tool observations can remain in the conversation history even after the underlying workspace state has changed, causing the agent to reason from stale context.

2.2 Why worth a general framework?

Volatile tool observations are not specific to files. They arise whenever an agent observes mutable external state through a tool. In production settings, such state may come from database records exposed by Postgres servers [4], issue and pull-request status from a GitHub server [6], or documents in a Slack workspace [11]. Each tool invocation gives the agent a snapshot of that state, and the snapshot may become outdated as soon as the underlying resource is modified. A resource-specific repair mechanism is not enough.

Our proposed abstraction treats tool observations as mutable context entries rather than immutable transcript items. Each entry is associated with a runtime-managed identifier and source metadata, such as the external resource it was derived from and the relevant content range. The runtime can then maintain an index from mutable sources to context entries and update the corresponding entries when source changes are observed. In this way, context validity is handled through a general runtime layer, while tool-specific logic is limited to registering observations and reporting source mutations.

3 CONCORD: A Context Coherence Framework for Volatile Agent Context

In this section, we present *CONCORD*, a context coherence framework that maintains the consistency between tool observations in agent context and the mutable sources from which they were derived. We first formalize the incoherence in volatile agent context and then describe CONCORD’s architecture, focusing on its core data structures and data paths.

3.1 Problem Formulation

We model an agent run as a sequence of steps. Before step t , the model-visible context contains the prompt and the previous action-observation history:

$$C_A^t = (p, a_1, o_1, \dots, a_{t-1}, o_{t-1}),$$

where p is the prompt, a_i is the tool action issued at step i , and o_i is the corresponding tool observation returned to the agent.

Let R denote an external resource, if an action a_i reads¹ R at step i , the agent records

$$o_i = \text{read}_i(R),$$

where $\text{read}_i(R)$ denotes the content returned by reading R at step i . Because R is mutable, a later read may return different

¹We use `read` as an abstract notation for any observation-producing access to an external resource, including file reads, database queries, or MCP resource requests.

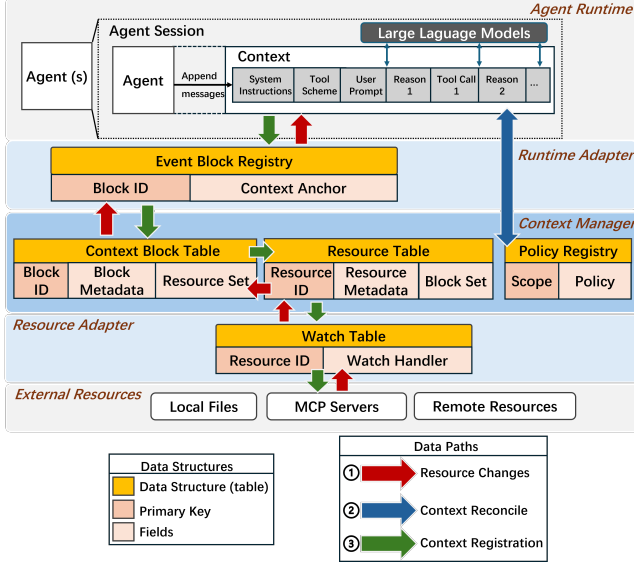


Figure 2. Architecture overview of CONCORD. A runtime- and source-agnostic *Context Manager* tracks tool observations as source-linked context blocks, connected to the agent runtime and to external resources through a *Runtime Adapter* and a *Resource Adapter*. Colored arrows denote the three data paths: block registration (green), source-change invalidation (red), and policy-driven context reconciliation (blue).

content:

$$\text{read}_j(R) \neq \text{read}_i(R), \quad j > i.$$

We say that observation o_i is **stale** at step j if it remains in the agent context but no longer matches the current result of reading the same resource:

$$\text{stale}(o_i, j) \iff o_i \in C_A^j \wedge o_i \neq \text{read}_j(R).$$

We define **incoherence in volatile agent context** as the condition in which the model-visible context contains at least one stale observation:

$$\text{volatile}(C_A^j) \iff \exists o_i \in C_A^j : \text{stale}(o_i, j).$$

A stale-observation failure occurs when the agent treats o_i as current evidence and produces an output that is inconsistent with $\text{read}_j(R)$. The research problem is therefore to efficiently detect and reconcile stale observations before they are reused in later model calls.

3.2 CONCORD Architecture

We address the aforementioned problem by introducing a context layer between the agent runtime and external resources. As shown in Figure 2, CONCORD is an extensible context-coherence framework organized around three components: a runtime adapter layer, a source-agnostic context manager, and a resource adapter layer.

A runtime adapter binds CONCORD to a concrete agent runtime. It observes tool results as they are inserted into the model-visible context C_A^t , maps the corresponding observations o_i to source-linked context blocks, and records an

anchor that identifies where each block appears in C_A^t . This boundary is necessary because different agent harnesses may store execution histories, tool observations, and LLM-facing messages in different internal representations.

The context manager is the core of the design. It maintains source-linked context blocks, tracks the relationship between blocks and mutable sources, records validity status, and applies reconciliation policies when sources change. Importantly, the context manager does not assume a specific runtime or resource type. It only requires that an observation can be associated with one or more source identifiers, and that source changes can be reported through a common event interface.

The resource adapter connects the context manager to external mutable sources. It normalizes source identifiers, obtains source versions, determines whether a source change overlaps with a recorded observation, and captures source-change events through hooks or polling. For example, a file adapter may track absolute paths, file versions, and line ranges, while an MCP or database adapter may track resource keys, timestamps, or version numbers. Thus, supporting a new resource type only requires implementing a new resource adapter, rather than changing the core context manager.

3.3 Tracking and Reconciling Context Blocks

Our context manager maintains context coherence through three steps: registration, invalidation, and reconciliation.

Context registration. When a tool observation is inserted into the agent context, the runtime adapter registers it as a source-linked context block. The block is assigned a stable block identifier and associated with the source metadata from which it was derived, including the source identifier, observed version, and relevant selector or range. The context manager then records this relation in both directions: from the block to its source set, and from each source to the blocks that depend on it. This bidirectional mapping is necessary because later updates arrive from the source side, while repairs must be applied to the corresponding context entries. The runtime adapter also records a context anchor for each block, allowing the repaired block to be written back to the runtime-specific conversation state.

Resource changes. When an external source changes, the resource adapter converts the resource-specific signal into a source-change event. For example, a file adapter may report a modified path, a new version stamp, and the changed range. The context manager uses the source-to-block mapping to locate all context blocks that may be affected, including blocks held by different agents. It then applies the block’s validity descriptor to decide whether the change actually invalidates the recorded observation. In the file case, this can be implemented as range-overlap checking: an edit only invalidates a block if the changed range overlaps with the range observed

by that block. This avoids treating every source update as a whole-resource invalidation.

Context reconciliation. After affected blocks are marked stale, the context manager reconciles them according to their repair policy. A policy may refresh the block by re-reading the source, annotate the block with a stale-context warning, suppress it from future prompts, or force the agent to re-read the source before continuing.

3.4 Extensible Adapter Interface and Policy Configuration

In order to be configured for different agent runtimes, resource types, and reconciliation requirements, CONCORD exposes three extension points.

Runtime extension. A runtime adapter defines the boundary between CONCORD and an agent runtime. Agent runtimes may organize execution history, context entries, and LLM-facing messages in different ways. CONCORD only requires the adapter to expose where tool observations enter the context and how repaired blocks can be reflected back into the model-visible state. Concretely, the adapter identifies tool results that should be registered, records a context anchor for each managed block, and renders repaired blocks back into the runtime’s context representation. This keeps the context manager independent of the runtime’s internal implementation.

Resource extension. A resource adapter defines how CONCORD observes mutable external sources. It specifies how source identifiers are normalized, how versions are obtained, how selected content is read, and how a source update is compared with a recorded observation. The key resource-specific component is the watch handle, which detects source changes and reports them to the context manager. Different deployments may choose different watch handles. For example, if a local file may be modified by arbitrary applications, the watch handle may need a system-level interception point, such as file-system notifications or instrumentation around write-related system calls, to observe updates outside the agent runtime. If the deployment only needs to track files modified by agents within a managed workspace, a narrower and cheaper handle can hook into the runtime’s file-write tool or workspace-mutation path. For remote resources, such as MCP servers or database-backed tools, the watch handle may rely on polling, version checks, server-side change notifications, or resource-specific subscription mechanisms. Therefore, each resource adapter defines a watch scope that matches the deployment’s required consistency guaranty. CONCORD then associates registered observations with the corresponding watch handle and reference-counts the handle across dependent blocks.

Policy configuration. When a block is marked stale, the context manager resolves a configurable repair policy from the policy registry. Policies can be configured globally, per

resource type, per source, or per block, and may also depend on the severity of the detected change. For example, if the refreshed source content is substantially different from the recorded observation, the policy may treat the old block as invalid and annotate, suppress, or require an explicit re-read. If the change is small and localized, the policy may directly refresh or patch the block’s model-visible payload. An eager refresh policy improves freshness by immediately re-reading the source, but may introduce additional latency or tool calls. This makes the correctness-overhead tradeoff explicit: deployments can choose stronger freshness guarantees at higher runtime cost, or cheaper policies with weaker guarantees.

4 Implementation and Evaluation

We implement CONCORD as a general framework for keeping agent context consistent with mutable external resources. To assess its effectiveness, we instantiate and evaluate one concrete use case: local files as the external resource.

4.1 Use Case Implementation

To apply CONCORD to a concrete setting, developers only need to implement adapters that connect a specific agent runtime and a specific external resource to the framework. In this use case, we implement a runtime adapter for the agent execution environment and a file-resource adapter for monitoring and re-reading local files. The file adapter treats each workspace path as a versioned source whose version is the SHA-256 of file contents, and uses `apply_patch` as the watch handle: any write that changes the content hash looks up tables in the context manager and runs reconciliation. For range observations, an anchored locator relocates the recorded span in the updated file and refreshes the block payload when relocation succeeds; otherwise the block is marked invalid. Whole-file structured observations are re-computed from the new content.

For the configurable reconciliation policy, we compare the old observation text with the refreshed content under a locality threshold: if the edit is local (same token length with at most $\max(2, \lfloor len/5 \rfloor)$ token substitutions, or sequence similarity ≥ 0.78), the agent-visible block is patched in place and kept valid; if the drift is larger, the block is left non-valid, omitted from prompt projection, and accompanied by a refresh notice that forces an explicit re-read.

4.2 Evaluation Setup

Benchmark. We evaluate on CONCORDBENCH-PIONEER, a 40-task pioneer version of our stale-observation benchmark derived from HotpotQA [28] multi-hop subset. Each HotpotQA task contains a question, a gold answer, a set of context documents, and `supporting_facts` needed to answer the question. We materialize the context documents as a local workspace. During construction, for each task, we select one

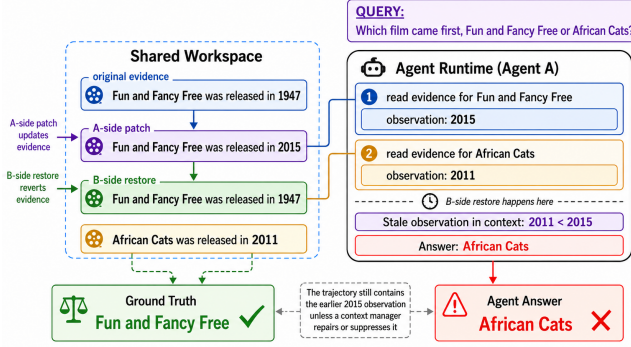


Figure 3. Example patch-and-restore race in CONCORDBENCH-PIONEER. The A-side patch makes the agent observe mutated evidence, while the B-side restore reverts the workspace; the agent fails if the stale observation remains in its trajectory.

or more supporting facts, and create a patch for each selected fact by replacing the original evidence sentence with a plausible mutated sentence. The mutation changes the factual content of the evidence, such as an entity, date, number, or comparison relation, so that an agent relying on the mutated observation is likely to derive an answer different from the original gold answer.

At evaluation time, each task starts from the mutated workspace W_{orig} and a patch set P . Before the agent runs, the benchmark applies the mutated patch and changes the workspace into a mutated workspace:

$$W_{\text{mut}} = \text{mutate}(W_{\text{orig}}, P).$$

The agent then interacts with W_{mut} using tools. When the agent reads a patched evidence span, the mutated text is returned by the tool and is recorded in the agent trajectory as a tool observation. Immediately after that tool observation has entered the trajectory, the race controller applies the B-side restore:

$$W_{\text{orig}} = \text{restore}(W_{\text{mut}}, P).$$

Thus, the workspace is again consistent with the original gold answer, but the agent trajectory may still contain the earlier mutated observation. The final answer is always graded against the original HotpotQA gold answer. Figure 3 shows one evaluation runtime example.

Agent harness. The default tool set is limited to `grep`, `glob`, `read_file`, and `answer`, excluding shell access and tests so that the benchmark measures stale-context reuse in the trajectory rather than independent environment inspection. The race controller primarily restores a patch after `read_file` returns the patched source; it also restores `grep`-hit sources if the agent moves on without reading them, and restores remaining patches before premature answer submission.

Baselines. We compare CONCORD with three baselines, plus an oracle upper bound:

- *Naive*: it leaves all prior tool observations unchanged in the context.

Table 1. Observed results on the 40-task HotpotQA-derived subset. Percentages are empirical recovery rates for this finite benchmark; † denotes the no-race oracle upper-bound setting.

	Recover Count ↑	Avg. Steps ↓	Tokens ↓
Claude Sonnet 5			
Oracle [†]	40/40 (100%)	3.03	285,417
Naive	20/40 (50.0%)	4.40	589,720
Alert	26/40 (65.0%)	4.33	609,750
Diff	36/40 (90.0%)	7.10	1,079,420
Concord	40/40 (100%)	4.03	502,596
GLM 5.2			
Oracle [†]	40/40 (100%)	3.70	219,753
Naive	15/40 (37.5%)	4.68	408,511
Alert	23/40 (57.5%)	4.65	430,712
Diff	31/40 (77.5%)	6.73	663,430
Concord	40/40 (100%)	4.75	389,953
DeepSeek V4 Pro			
Oracle [†]	40/40 (100%)	4.58	410,820
Naive	24/40 (60.0%)	5.60	562,851
Alert	31/40 (77.5%)	5.83	621,379
Diff	38/40 (95.0%)	8.20	903,032
Concord	40/40 (100%)	5.38	524,862

- *Alert*: it appends `[alert] path is modified` to the next tool output after a restore (and to the last tool message if the agent tries to answer early), leaving correction to the agent.
- *Diff*: it adds a diff tool that reports restored patches as `- mutated / + original` and blocks answer until the agent has seen a fresh diff for the current workspace generation.
- *Oracle*: it runs the same agent and tools on the original workspace with no race injection, serving as an upper bound that isolates HotpotQA difficulty from stale-observation failures.

Evaluation metrics. We report three evaluation metrics:

- *Recover count* is the number of tasks in which the final answer matches the original gold answer after the source has been restored; it is our primary measure of whether the system avoids stale-context hallucination.
- *Average steps* measures the number of agent events required to finish the task, where fewer steps indicate lower recovery overhead.
- *Token consumption* measures total token usage and captures the prompt-level cost of recovery.

4.3 Results and Analysis

Table 1 reports recover count, average steps, and token consumption across three frontier models. Under the constructed patch-and-restore race conditions, Concord also achieves a recover count of 40/40 for each model and uses fewer tokens

than Naive, Alert, and Diff in the evaluated runs. Concord additionally requires fewer steps than Diff in all three settings. These results indicate a better observed accuracy-cost tradeoff for runtime-level context repair than for prompt-level recovery on ConcordBench-Pioneer; they should not be interpreted as a workload-independent guarantee of perfect recovery. To better understand these observed differences, we next conduct a trace-level analysis using the complete GLM 5.2 runs.

Stale evidence persists inside the trajectory. Naive recovers only 15/40 tasks in the traced run. The dominant failure mode is short and direct: the agent reads mutated evidence, forms an answer, and later repeats that answer after the benchmark restores the file. Many wrong predictions match the mutated value rather than the gold answer, such as a changed number, year, or entity name. Even when the runtime intercepts premature answers and restores remaining patches (28/40 Naive tasks), the agent receives no signal that its earlier observation is stale, so the restored workspace does not repair the model-visible trajectory.

Prompt-level signals trade accuracy for cost. Alert improves the performance from 15/40 to 23/40, but only when the notice leads to re-grounding. If an alert is followed by a later `read_file`, the agent is correct in 17/18 cases; without a later read, it is correct in only 2/8 cases when the alert is visible and 4/14 cases when the alert is only appended around answer interception. Diff gives a stronger signal by showing an explicit before/after hunk, reaching 31/40. However, it turns recovery into a tool protocol: Diff uses 269 events and 663k tokens, compared with 148 events and 220k tokens for Oracle, and 21/40 tasks call `diff` at least twice because of freshness gating.

Runtime-level reconciliation repairs stale observations in the evaluated traces. CONCORD achieves a recover count of 40/40 in the evaluated GLM 5.2 run while consuming 390k tokens, fewer than Naive, Alert, and Diff in this setting. It also needs fewer reads and steps than the strongest prompt-level baseline: 1.30 `read_file` calls per task and 4.75 average steps, versus 2.25 reads per task and 6.73 steps for Diff. The reason is that CONCORD patches local stale blocks in place, or removes invalid blocks from prompt projection and forces a re-read when drift is larger. The next model call therefore sees repaired evidence or an explicit refresh requirement, rather than a stale observation plus an advisory warning.

Within ConcordBench-Pioneer, the evaluated methods show a consistent ordering: alerts are inexpensive but incomplete, diffs recover more evaluated cases at higher cost, and runtime context reconciliation matches the oracle recover count while using fewer tokens than the prompt-level race baselines.

4.4 Limitations and Failure Modes

Our evaluation covers 40 HotpotQA-derived tasks, local files, a restricted tool harness, and a controlled patch-and-restore

race. Therefore, the observed 40/40 recover count for each model does not imply perfect or workload-independent reliability. The current prototype monitors writes routed through `apply_patch`; out-of-band changes require broader change-detection mechanisms, while polling may miss rapid updates. CONCORD also relies on correct source identifiers, versions, and source-to-observation mappings. Large rewrites, ambiguous span relocation, multi-source observations, or repeated updates may force re-reading, increasing latency and token cost. Moreover, CONCORD does not revise summaries, plans, or decisions derived from stale observations. We leave evaluation on broader workloads, concurrent agents, repeated update cycles, and non-file resources to future work.

5 Related Work

Consistency in Multi-Agent Workspaces. Prior work has studied consistency in multi-agent workspaces mainly at the system or runtime level. STORM [12] prevents stale writes through local state consistency checks; CAID [5] isolates agents in separate workspaces and resolves conflicts at merge time; and CodeCRDT [20] explores convergence for shared code state. Other systems manage state through workspace snapshots or explicit runtime boundaries, such as InfiAgent [29] and LemonHarness [21]. Recent studies also formalize stale-generation and related concurrency anomalies [10], or show that stale environmental observations can cause grounding failures [22]. These works primarily manage workspace state, write operations, or snapshot reconstruction. In contrast, we study prompt-time coherence by tracking observations and handling source changes through invalidation, refresh, or annotation.

Agent Memory Systems. This line of work studies how agents manage long-term memory and maintain the freshness of stored facts. MemGPT [18] introduces an OS-inspired memory hierarchy for managing limited context windows; Self-GC [8] treats context as indexed objects with lifecycle operations such as folding, masking, and pruning; and VISTA [27] represents working memory as typed and addressable blocks. Recent work also examines memory staleness and temporal validity: STALE [3] evaluates whether agents can recognize when prior memories become invalid; Governed Shared Memory [13] studies stale propagation and provenance collapse in shared agent memory; and Engram [25] and Supersede [19] use temporal validity or super-session chains to address delayed memory updates. These systems are complementary to ours: they mainly focus on retrieval-time evidence selection, memory capacity management, or conversational fact updates, whereas we track tool observations that have already been materialized in an agent’s private prompt context.

Cache Coherence in Computer Systems. The consistency problem between agent context and external state can be naturally related to cache coherence in classical computer systems. From this perspective, mutable external resources, such as file systems or MCP services, serve as the source of truth, while tool observations recorded in an agent’s prompt act as cached copies of that state. Traditional cache-coherence mechanisms [1, 7, 23] maintain consistency between cached copies and the source state through techniques such as versioning, invalidation, and update propagation. These ideas provide useful guidance for agent context management, such as maintaining source metadata, dependency information, and invalidation policies for context entries.

6 Conclusion and Future Work

In this work, we identify and formally define incoherence in volatile agent context. To address the problem, we propose CONCORD, a context coherence framework that maintains the consistency between tool observations in agent context and the mutable external resources. CONCORD is applicable across different agent runtimes and external resources, and can be easily extended to new runtime-resource settings.

Several directions remain for future work. First, while our current use case focuses on file-based tool interactions, it can be extended to broader classes of mutable resources (e.g. MCP Tools). Second, our current implementation focuses on tool observations, but agents often produce summaries, plans, assumptions, or decisions that are derived from those observations. Tracking the validity of such derived context is a harder problem and requires richer context dependency modeling. Finally, regarding the reconciliation policy, we could explore more efficient strategies, such as finer-grained replacement, batched propagation, or lazy updates, to reduce runtime overhead and avoid unnecessary modifications to the agent context.

References

- [1] Sarita V Adve and Kourosh Gharachorloo. Shared memory consistency models: A tutorial. *computer*, 29(12):66–76, 1996.
- [2] Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, November 2024. Accessed: 2026-07-08.
- [3] Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. Stale: Can llm agents know when their memories are no longer valid? *arXiv preprint arXiv:2605.06527*, 2026.
- [4] crystaldba. postgres-mcp: Postgresql model context protocol server. <https://github.com/crystaldba/postgres-mcp>, 2024. Accessed: 2026-07-08.
- [5] Jiayi Geng and Graham Neubig. Effective strategies for asynchronous software engineering agents. *arXiv preprint arXiv:2603.21489*, 2026.
- [6] GitHub. github-mcp-server: Github model context protocol server. <https://github.com/github/github-mcp-server>, 2024. Accessed: 2026-07-08.
- [7] James R Goodman. Using cache memory to reduce processor-memory traffic. In *Proceedings of the 10th annual international symposium on Computer architecture*, pages 124–131, 1983.
- [8] Xubin Hao, Hongjin Meng, Xin Yin, Jiawei Zhu, and Chenpeng Cao. Self-gc: Self-governing context for long-horizon llm agents. *arXiv preprint arXiv:2607.00692*, 2026.
- [9] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, pages 23247–23275, 2024.
- [10] Sajjad Khan. Verified detection and prevention of concurrency anomalies in multi-agent large language model systems. *arXiv preprint arXiv:2606.17182*, 2026.
- [11] korotovskiy. slack-mcp-server: Slack model context protocol server. <https://github.com/korotovskiy/slack-mcp-server>, 2024. Accessed: 2026-07-08.
- [12] Mengyang Liu, Taozhi Chen, Zhenhua Xu, Xue Jiang, and Yihong Dong. Multi-agent collaboration with state management. *arXiv preprint arXiv:2605.20563*, 2026.
- [13] Yanki Margalit, Nirit Cohen-Inger, Erni Avram, Ran Taig, and Oded Margalit. Governed shared memory for multi-agent llm systems. *arXiv preprint arXiv:2606.24535*, 2026.
- [14] OpenClaw Contributors. OpenClaw Documentation: Context. <https://docs.openclaw.ai/concepts/context>, 2026. Accessed: 2026-07-08.
- [15] OpenClaw Contributors. OpenClaw Documentation: Tools. <https://github.com/openclaw/openclaw/blob/main/docs/tools/index.md>, 2026. Accessed: 2026-07-08.
- [16] OpenCode Contributors. OpenCode: A Powerful AI Coding Agent. <https://github.com/opencode-ai/opencode>, 2026. Accessed: 2026-07-08.
- [17] OpenCode Contributors. OpenCode Documentation: MCP Servers. <https://opencode.ai/docs/mcp-servers/>, 2026. Accessed: 2026-07-08.
- [18] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: towards llms as operating systems. 2023.
- [19] Vedant Patel. Supersede: Diagnosing and training the memory-update gap in llm agents. *arXiv preprint arXiv:2606.27472*, 2026.
- [20] Sergey Pugachev. Codecrdt: Observation-driven coordination for multi-agent llm code generation. *arXiv preprint arXiv:2510.18893*, 2025.
- [21] Kailong Ren, Fubo Sun, Jiachen Liu, Liu Yang, Zimo Yin, Jiaying Li, Congli Yin, Ming He, Yu Huo, Jiawei Liu, et al. Lemonharness technical report. *arXiv preprint arXiv:2606.24311*, 2026.
- [22] Strick Sheng, Ziyue Wang, and Liyi Zhou. When agents overtrust environmental evidence: An extensible agentic framework for benchmarking evidence-grounding defects in llm agents. *arXiv preprint*

- arXiv:2605.08828*, 2026.
- [23] Daniel Sorin, Mark Hill, and David Wood. *A primer on memory consistency and cache coherence*. Morgan & Claypool Publishers, 2011.
 - [24] Chenyu Wang, Yueyuan Li, Yingmin Liu, and Yang Shu. Conflictrag: Detecting and resolving knowledge conflicts in retrieval augmented generation. *arXiv preprint arXiv:2605.17301*, 2026.
 - [25] Liuyin Wang. Less context, more accuracy: A bi-temporal memory engine for llm agents where a lean retrieved context beats the full history. *arXiv preprint arXiv:2606.09900*, 2026.
 - [26] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919, 2025.
 - [27] Binyan Xu, Haitao Li, and Kehuan Zhang. Llm agents are latent context managers: Eliciting self-managed context via a proprioceptive dashboard. *arXiv preprint arXiv:2606.30005*, 2026.
 - [28] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380, 2018.
 - [29] Chenglin Yu, Yuchen Wang, Songmiao Wang, Hongxia Yang, and Li Ming. Infiagent: An infinite-horizon framework for general-purpose autonomous agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 35884–35894, 2026.